WEEK 1 · DAY 1

Foundry Architecture for Enterprise AI

Data Products, Ontology-First Design & Domain-Driven Modeling

Rajesh Pasham

Program Context

How today's session fits the wider 8-week delivery

Session Format	One combined live lecture for India & Europe — 2:00–3:00 PM IST / 10:30–11:30 AM CEST
Platform	Individual Palantir Developer Tier accounts, used from Day 3 onward — today is concepts and architecture only
Shared Domain, Distinct Use Cases	All batches work in Insurance. Today's Customer/Policy/Claim example is the shared reference model each batch will apply to its own assigned use case
Use Case Finalization	Today's use case allocation is the trainer's starting point — confirmed with participants during Week 1

Today's Roadmap

Six topics, building on each other in order

1	**Data Products**	What every dataset and model in Foundry is packaged as
2	**Ontology-First Design**	Why we model the business before we build pipelines
3	**Domain-Driven Ontology Modeling**	Turning business language into a defensible model
4	**Advanced Object Modeling**	Properties, Shared Properties, Link Types, Action Types
5	**Branching Strategies**	Changing the model safely, without breaking production
6	**Incremental Processing**	Keeping the Ontology fresh without full rebuilds

By the End of Today

Six ideas that everything else in this program builds on

Explain what a Data Product is

And why Foundry treats every dataset and model this way

Explain why we design Ontology-first

Not pipelines-first, not dashboards-first

Model a domain using Domain-Driven Design

Turning business language into Object Types

Use branching to isolate in-progress modeling work

Without breaking what's already in production

How This Session Works

A concepts lecture today — hands-on begins Day 3

Today (Day 1–2)

- Concepts and architecture, explained through this lecture and diagrams
- No live Palantir account work — every example is presented on slides
- You'll leave with the reasoning, not yet the muscle memory

Day 3 Onward

- Self-paced hands-on lab in your own Developer Tier account
- You build the exact model discussed today, for real
- Day 4–5: mentoring and review on what you built

TOPIC 1

Data Products

What every dataset and model in Foundry is packaged as

What Is a Data Product?

The foundation every later topic today builds on

A Data Product is a governed dataset or model, exposed through standard output ports — such as an API or the Ontology — so it can be discovered and consumed without anyone needing to know how it was built.

This is the packaging concept that everything else this week assumes — an Ontology is only as trustworthy as the Data Products feeding it.

Anatomy of a Data Product

Four elements that make a dataset discoverable and trustworthy

Producer	A team builds and owns a dataset or model — for example, Claims
Output Ports	The product is exposed via an API and/or the Ontology, not a raw table
Discoverability	Other teams find it through Data Catalog or Ontology Manager, not a message to a colleague
Consumer	Downstream teams build on top of the product without touching its internals

This is why Data Products scale: independent teams evolve their own pipelines without breaking each other's consumers, as long as the output contract stays stable — and it's the prerequisite everything else this week assumes.

TOPIC 2

Ontology-First Design

Why we model the business before we build pipelines

Two Ways to Start

The order you build in shapes everything that follows

Pipeline-First (the old way)

Build ETL first, figure out the data model later. Every consumer re-derives their own definition of “a claim.”

Ontology-First (this program)

Agree on what a Claim, Policy, and Customer are — once — then build pipelines that feed that shared model.

The Ontology as Digital Twin

What “Ontology-first” actually buys you

One shared definition, everywhere

Every application, Copilot, and Agent built later reasons against the same Customer, Policy, and Claim

Relationships, not just records

The Ontology captures how Customer, Policy, and Claim connect — not just what each looks like alone

The layer AI actually reasons over

A Copilot doesn't read raw tables — it reasons over the Ontology's objects and relationships

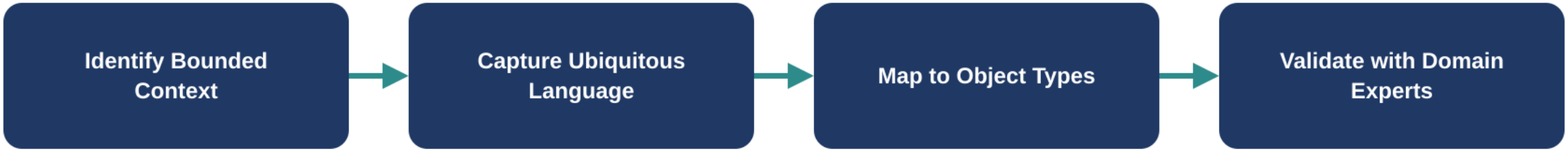
TOPIC 3

Domain-Driven Ontology Modeling

Turning business language into a defensible model

Borrowing from Domain-Driven Design

A four-step process for deciding what becomes an Object Type



Each step is covered on its own over the next few slides — skipping straight to Object Types without the first two steps is the most common modeling mistake.

Step 1 — Identify the Bounded Context

Where does one word start meaning something different?

Claims processing and Underwriting may use the same word — “risk” — to mean two different things. Model them as separate bounded contexts rather than forcing one shared definition that satisfies neither.

A bounded context is a boundary within which a term has exactly one meaning. Crossing that boundary without realizing it is where Ontology drift usually starts.

Step 2 — Capture the Ubiquitous Language

Model the words domain experts actually use

Interview, don't infer

Get the exact terms an adjuster or underwriter uses — not the column names in a source database

Source systems lie about language

A column called cust_typ_cd tells you nothing about what the business calls that concept

Write the glossary down

A shared glossary of terms is a modeling artifact, not an afterthought

Steps 3–4 — Map to Object Types, Then Validate

Where the sentence becomes the model — and gets checked

"A claim has a policy, a claimant, and a status."

becomes → Claim, Policy, and Customer as Object Types, with status modeled as a property on Claim.

Step 4 — The recognition test

An adjuster or underwriter should look at the model and say "yes, that's how we talk about claims"

Expect pushback, and use it

Disagreement here is far cheaper to resolve than after Week 3's Actions are built on the wrong model

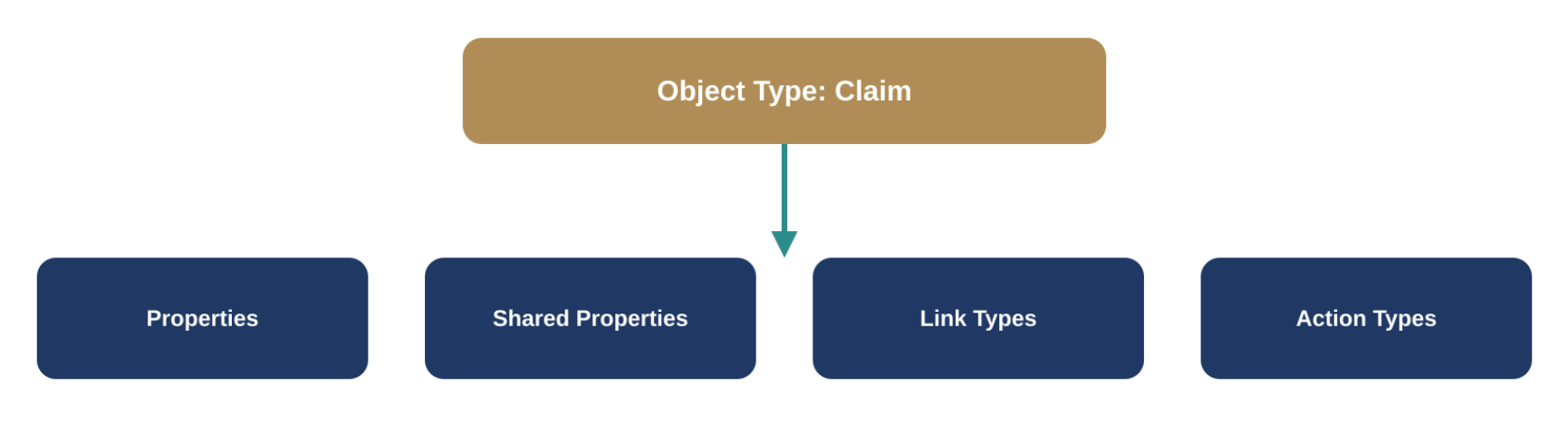
TOPIC 4

Advanced Object Modeling

What a well-designed Object Type is actually made of

Anatomy of an Object Type

Four elements, working together



Properties vs. Shared Properties

The distinction that prevents the most common source of drift

Properties

Attributes specific to one Object Type — claimAmount and fraudScore belong to Claim and nowhere else.

Shared Properties

Centrally managed attributes reused consistently across Object Types — createdAt, lastModifiedBy — defined once.

Every team inventing its own version of "created date" is one of the most common, and most avoidable, sources of Ontology drift.

Link Types — Modeling Relationships

The schema of how Object Types connect

A Link Type is a relationship's schema

Claim links to Policy; Policy links to Customer — defined once, navigable everywhere

Cardinality is a fact, not a guess

“A customer can hold more than one policy” is a direct statement of cardinality — model it as stated, don't infer

This is what makes an Ontology navigable

Without Link Types, an Agent in Week 5 has no way to move from a Claim to its Customer

Action Types — Modeling Change

The schema for how an Object Type is allowed to change

What It Defines	Parameters, submission criteria, and the exact side effects a change to an object can have
Why It Matters Later	Week 3's writeback capability is built entirely on Action Types defined this week
The Governance Connection	An Action Type is where “who can change this, and under what conditions” gets encoded into the model itself

TOPIC 5

Branching Strategies

Changing the model safely, without breaking production

Why Branch?

The problem branching solves

Modeling changes directly against production risks breaking every application, Copilot, and Agent already relying on the current model — a branch is an isolated space to get the change right first.

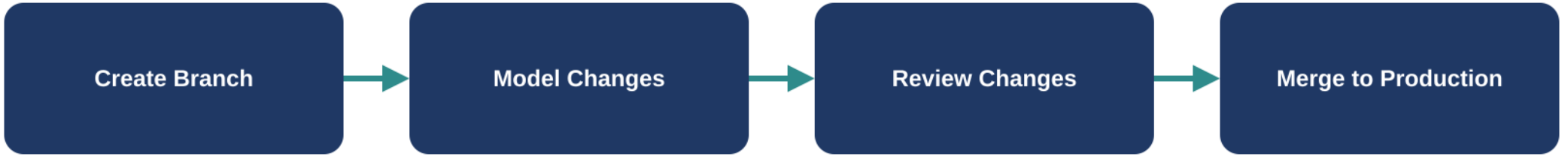
Feature Branches vs. Environment Branches

Two patterns, two different purposes

Feature Branch	Model a new Object Type or property change in isolation, then merge once it's validated
Environment Branch	Separate development and staging Ontology states before promoting to production
Short-Lived by Design	Branches are meant to be merged quickly — long-lived branches drift and become painful to reconcile

The Branch Lifecycle

The same four steps, every time



This is the workflow you'll actually run in your Developer Tier account from Day 3 — today is about understanding why each step exists.

TOPIC 6

Incremental Processing

Keeping the Ontology fresh without full rebuilds

Full Rebuild vs. Incremental Processing

Two ways to keep a pipeline's output current

Full Rebuild

Reprocesses the entire dataset on every run. Simple to reason about, but slow and expensive as data grows.

Incremental Processing

Processes only new or changed records since the last run. The standard pattern for production-scale pipelines.

Conceptually, incremental processing looks like this — the hands-on build comes later in the program:

When to Use Each Approach

A judgment call, not a default

Favor Full Rebuild When	Data volume is small, logic changes frequently during early development, or correctness matters more than speed
Favor Incremental When	Data volume is large, the pipeline runs on a schedule, or freshness at production scale is the goal
The Program's Default	This program treats incremental processing as the standard pattern from Week 1 onward, matching production practice

Putting It All Together

Today's six topics as one architecture

Source Systems → Data Products	Claims, policy, and customer systems generate data, packaged and exposed as governed Data Products
Data Products → Ontology	Object Types, Link Types, and Action Types, modeled domain-first using the four-step DDD process
Ontology → Branches → Enterprise Consumers	Changes evolve safely on branches before merging, feeding the Copilots and Agents built in later weeks

Recap & What's Next

What We Covered Today

- Data Products: governed, discoverable outputs
- Ontology-first design over pipeline-first
- Domain-Driven modeling: language → Object Types
- Object anatomy: Properties, Links, Actions
- Branching for safe, isolated modeling
- Incremental processing over full rebuilds

Day 2 — Governance First

- Roles and Markings access control
- Security policies and data lineage
- Audit trails on Ontology changes
- Enterprise architecture patterns
- Also a concepts lecture — no live platform work

Questions

Rajesh Pasham